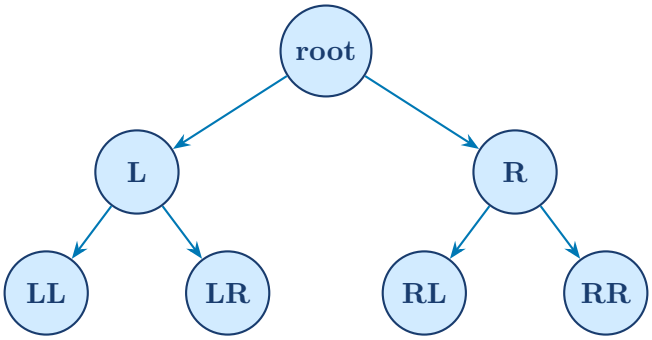
ADDS Lab Work

Algorithms and Dynamic Data Structures



Technical Documentation

Set-Theoretic Operations on Textual Data
using Binary Search Trees

Module: Algorithms and Dynamic Data Structures

Document type: Lab Project Technical Report

Language: C (C99 Standard)

Contents

1 Project Overview and Core Objective

This document presents the technical design and implementation of a high-performance application developed as part of the Algorithms and Dynamic Data Structures (ADDS) course. The program applies mathematical set theory — specifically the operations of **Union**, **Intersection**, and **Difference** — to textual data extracted from multiple input files.

The system is designed to:

1. Accept multiple plain-text files as input.
2. Parse each file into its constituent paragraphs and further decompose each paragraph into individual phrases.
3. Represent each paragraph as a **Binary Search Tree (BST)** of phrases, stored within a dynamically allocated array indexed by paragraph number.
4. Efficiently compute set operations across selected paragraphs from different files.

By organising textual data in BSTs, the program benefits from $O(\log n)$ search complexity, enabling fast comparisons required by the set operations.

The three supported operations are defined as follows:

Operation	Description
Union	Produces all unique phrases found across all selected paragraphs.
Intersection	Produces only the phrases present in every selected paragraph simultaneously.
Difference	Produces phrases present in one selected paragraph but absent in another.

2 Implementation Architecture

2.1 Modular File Organisation

The codebase follows a strict modular design pattern: each source file encapsulates a single, well-defined area of responsibility. This separation of concerns promotes maintainability, clarity of purpose, and independent testing of components.

2.1.1 Header Files

File	Responsibility
`00_structures.h`	Defines all data structure types used across the application.
`01_text_processing.h`	Declares the function prototypes for text parsing and manipulation.
`02_operations.h`	Declares the function prototypes for set operations (union, intersection, difference).
`03_interface.h`	Declares the function prototypes for the user-interface and menu display.

2.1.2 Source Files

File	Responsibility
`04_interface.c`	Implements the user-facing menu and interaction prompts.
`05_file_processing.c`	Implements file opening, reading, and loading into memory.
`06_text_processing.c`	Implements paragraph and phrase extraction logic.
`07_main.c`	Entry point; coordinates user input, function dispatch, and the overall program flow.
`08_union.c`	Implements the Union set operation.
`09_intersection.c`	Implements the Intersection set operation.
`10_difference.c`	Implements the Difference set operation.

3 Data Structures

3.1 Design Rationale

The system uses two primary data structures in composition: a **dynamic array** of paragraph entries and a **Binary Search Tree (BST)** per paragraph. This combination was deliberately chosen to satisfy two competing requirements simultaneously:

- **Random access to paragraphs in $O(1)$:** The dynamic array allows any paragraph to be retrieved by its index in constant time, which is critical for efficiently selecting paragraphs as operands for set operations.
- **Efficient phrase search, insertion, and deletion in $O(\log n)$:** Storing phrases in a BST allows the comparison operations required by Union, Intersection, and Difference to be performed without exhaustive linear scans.

Memory is allocated dynamically and grows only as required; no memory is wasted on pre-allocated capacity beyond what the parsed data demands.

3.2 The `tree_node` Structure

Each node of a paragraph's BST is represented by the following structure:

```
typedef struct tree_node
{
    char          *phrase;   /* The phrase/sentence stored
       at this node */
    struct tree_node *lc;    /* Pointer to the left child
                        */
    struct tree_node *rc;    /* Pointer to the right child
                       */
} tree_node;
```

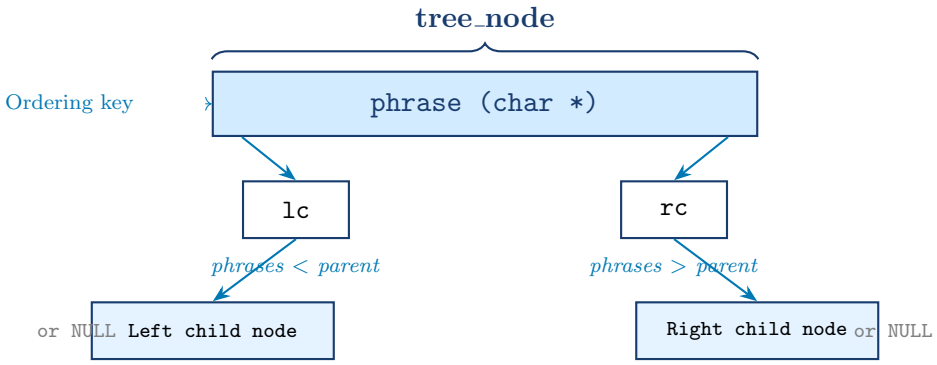
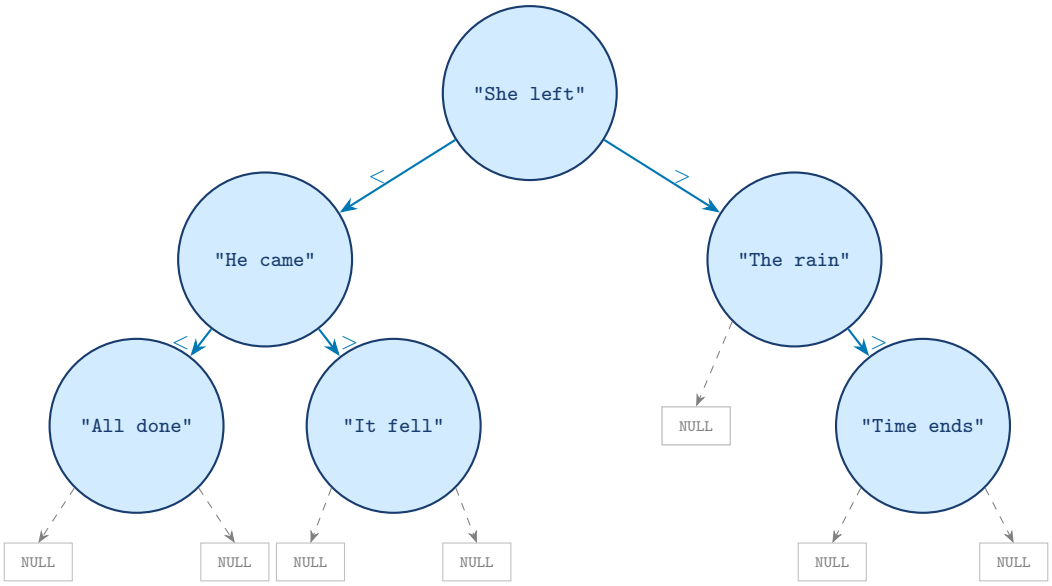


Figure 1: Memory layout and semantic role of a single `tree_node`.

The ordering invariant of the BST is enforced using lexicographic comparison (`strcmp`): a phrase strictly less than the current node's phrase is placed in the left subtree; a phrase strictly greater is placed in the right subtree. Duplicate phrases are not inserted.

3.3 Binary Search Tree — Example

The figure below illustrates how a sample paragraph is stored as a BST after all its phrases have been inserted in the order they appear in the text.



Phrases in **left** subtree are lexicographically **smaller** than the parent node.

Phrases in **right** subtree are lexicographically **larger** than the parent node.

Figure 2: A BST representing one paragraph. Nodes are ordered by `strcmp` on the phrase strings.

3.4 The `text_file` Structure

A complete text file is represented by the following structure:

```
typedef struct text_file
{
    tree_node **paragraph_array;  /* Dynamic array of BST
       root pointers     */
    int          paragraph_count;  /* Number of paragraphs
       in the file       */
    char        *file_start;       /* Pointer to the raw
       file buffer (heap) */
    char        *index;            /* Navigation cursor
       inside the buffer    */
} text_file;
```

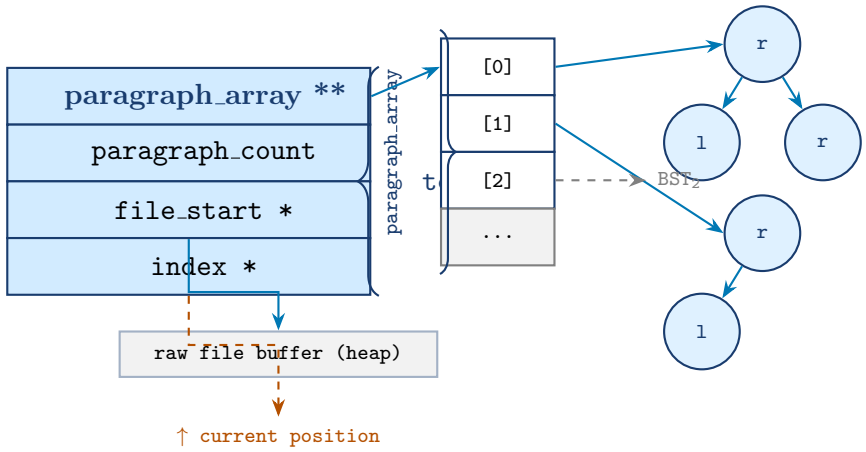
The four fields serve distinct roles:

- `paragraph_array`: A dynamically reallocated array of `tree_node*` pointers. Each element at index i is the root of the BST representing the i-th paragraph. This array grows via `realloc` as new paragraphs are discovered during parsing.
- `paragraph_count`: An integer counter updated after each paragraph is fully

extracted, recording the total number of paragraphs processed.

- `file_start`: A pointer retained at the beginning of the heap-allocated file buffer, used exclusively to free the buffer when the structure is deallocated.
- `index`: A cursor pointer that advances through the buffer character by character during parsing. It is passed by value to extraction functions so that those functions return an updated position.

3.4.1 Structural Overview of `text_file`



Each `paragraph_array[i]` is a pointer to the root of the BST for paragraph i.

Figure 3: Composite memory layout of a `text_file` instance showing the dynamic paragraph array and its relationship to individual BST paragraphs.

3.5 Text Delimiters

The parsing logic relies on two categories of delimiter:

Delimiter	Character(s)	Semantic meaning
Phrase delimiter	`. ? ! ;`	End of a phrase/sentence
Paragraph delimiter	`\n` (newline)	End of a paragraph

4 The Abstract Machine — Core Functions

This section describes each fundamental function that constitutes the primitive operations of the abstract machine on which the higher-level set operations are built.

4.1 Node Construction

```
tree_node *create_phrase(char *phrase)
```

Role: Allocates and initialises a new BST leaf node containing the given phrase.

Algorithm:

1. Allocate a new `tree_node` on the heap.
2. Duplicate the input string using `strdup()` and assign it to the `phrase` field.
3. Set both child pointers (`lc`, `rc`) to `NULL`.
4. Return a pointer to the newly created node.

Complexity: $O(1)$

4.2 BST Insertion

```
void insert_copy(tree_node **r, char *phrase)
```

Role: Inserts a phrase into a BST while preserving the BST ordering invariant.

Algorithm:

1. Traverse the tree recursively using the standard BST search strategy.
2. At each node, compare the input phrase to the current node's phrase using `strcmp()`.
3. Proceed left if the input phrase is lexicographically smaller; proceed right if it is larger.
4. Upon reaching a `NULL` position, call `create_phrase()` to construct and attach the new node.

Complexity: $O(\log n)$ average case; $O(n)$ worst case (degenerate tree).

4.3 BST Search

```
int find_phrase(tree_node *paragraph, char *phrase)
```

Role: Searches for a specific phrase within a paragraph's BST.

Algorithm:

1. Exploit the BST ordering property to navigate directly toward the target phrase without examining unrelated branches.
2. Return `1` (true) if the phrase is found; return `0` (false) otherwise.

Complexity: $O(\log n)$ average case.

4.4 In-Order Display

```
void display_inorder(tree_node *r)
```

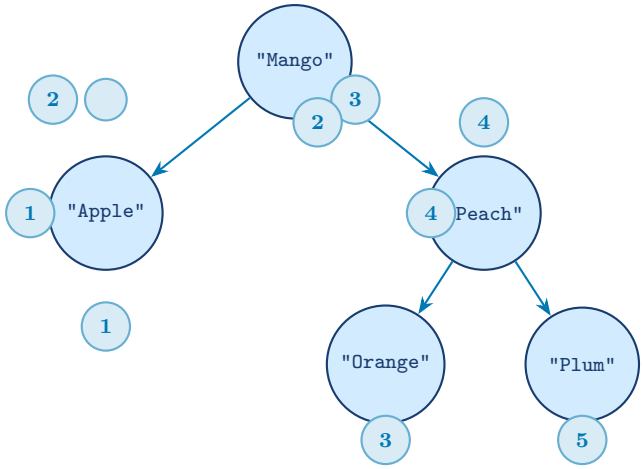
Role: Traverses and displays all phrases stored in a BST in lexicographic order.

Algorithm:

1. **Null guard:** If the current node is `NULL`, return immediately.
2. **Left subtree:** Recursively call the function on the left child to process all phrases smaller than the current one.
3. **Output:** Print the phrase stored at the current node, followed by a newline.
4. **Right subtree:** Recursively call the function on the right child to process all phrases larger than the current one.

Complexity: $O(n)$

4.4.1 Traversal Order Illustration



In-order visit sequence: **Apple** → **Mango** → **Orange** → **Peach** → **Plum**

Figure 4: In-order traversal produces phrases in ascending lexicographic order. Circled numbers indicate the visitation sequence.

4.5 Memory Deallocation

4.5.1 `void free_tree(tree_node **r)`

Role: Systematically deallocates all heap memory occupied by a BST, including both the structural nodes and their dynamically allocated string payloads.

Algorithm: The function employs a recursive **post-order traversal**, ensuring that all descendants of a node are fully deallocated before the node itself is addressed.

For each node visited:

1. Recursively free the left subtree.
2. Recursively free the right subtree.
3. Release the memory allocated for the `phrase` string (payload).
4. Free the node structure itself and assign `NULL` to the pointer to prevent dangling pointer vulnerabilities.

Result: Complete and safe reclamation of all memory associated with the tree, with no memory leaks from either the string payloads or the node structures.

4.5.2 `void free_struct(text_file *file)`

Role: Comprehensively releases all dynamically allocated memory components associated with a `text_file` instance.

Algorithm:

1. **Nested deallocation:** Iterate through the `paragraph_array` and invoke `free_tree()` on each BST to clear all parsed phrase data.
2. **Array reclaim:** Deallocate the `paragraph_array` memory block (the array of pointers).
3. **Buffer release:** Free the `file_start` buffer containing the raw file contents.

Result: Total deallocation of the `text_file` instance, returning all hierarchically allocated heap memory to the operating system.

5 Text Processing

5.1 `extract_phrase(char *index)`

Role: Isolates a single phrase from a continuous stream of characters, using punctuation delimiters to determine phrase boundaries.

Algorithm:

1. **Leading cleanup:** Advance the `index` pointer past any leading spaces, tabs, or delimiter characters (`.`, `?`, `!`, `;`) to ensure that reading begins at the first meaningful character.
2. **Dynamic extraction:** Read characters one by one, appending each to a dynamically resizing heap buffer (managed via `realloc`). This loop continues

until a phrase delimiter, a newline character, or the end of the file is encountered.

3. **Trailing cleanup:** Remove any trailing whitespace from the extracted buffer.
4. **String termination:** Append a null terminator (`\0`) to produce a valid C-string.

Return value: A pointer to a dynamically allocated string containing the clean, isolated phrase. The `index` pointer is advanced to the character immediately following the extracted phrase. If no valid text is found (e.g. the pointer is already at a paragraph boundary or the end of file), the function returns `NULL`.

5.1.1 Phrase Extraction Process — Visualisation

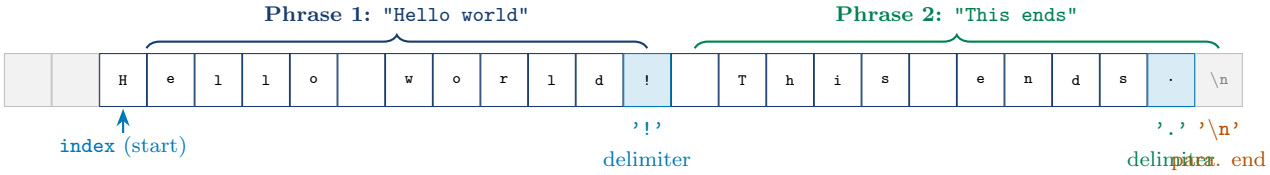


Figure 5: Character-level view of how `extract_phrase()` segments a raw buffer into individual phrases using delimiter characters.

6 Set Operations

All three set operations follow a consistent two-tier design:

- A **builder function** that performs the actual tree traversal and selective insertion into a new result tree.
- A **wrapper function** that handles empty-tree edge cases and returns the completed result tree.

6.1 Union

Definition: $P_1 \cup P_2 = \{x \mid x \in P_1 \vee x \in P_2\}$

The Union operation produces a new BST containing every unique phrase present in either of the two input paragraphs.

6.1.1 `void merge_tree(tree_node **r1, tree_node *r2)`

Role: Merges the contents of tree `r2` into tree `r1`.

Algorithm: Perform an in-order traversal of `r2`, and for each phrase encountered, call `insert_copy()` to insert it into `r1`. The BST property of `r1` ensures that duplicate phrases are naturally rejected.

Result: `r1` contains all unique phrases from both trees in correct lexicographic

order.

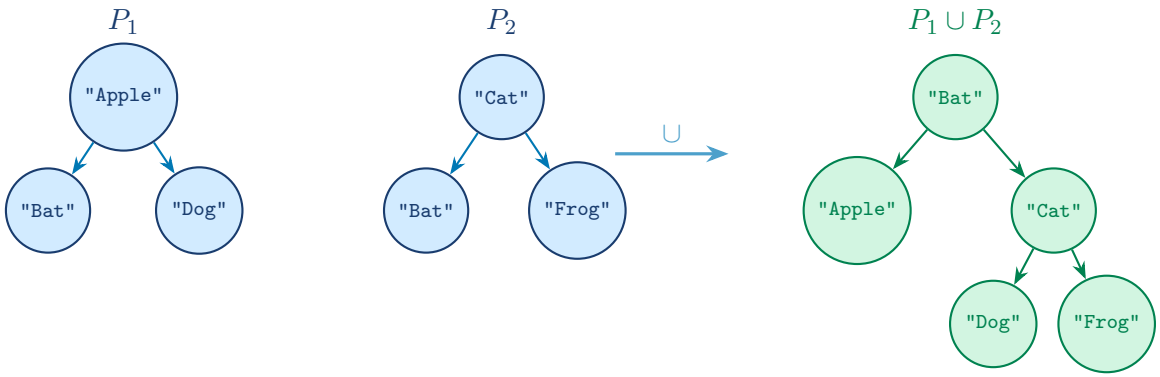
6.1.2 `tree_node *union_op(tree_node *paragraph1, tree_node *paragraph2)`

Role: Performs the full Union operation between two paragraphs.

Algorithm:

1. Create an empty **union tree**.
2. Call `merge_tree()` to merge `paragraph1` into the union tree.
3. Call `merge_tree()` again to merge `paragraph2` into the union tree.
4. Return the union tree.

6.1.3 Union — Visual Illustration



Note: "Bat" appears in both P_1 and P_2; it is inserted only once (BST rejects duplicates).

Figure 6: Union of two paragraph BSTs. Phrases from P_2 are merged into a copy of P_1; duplicates are automatically excluded by the BST insertion property.

6.2 Intersection

Definition: $P_1 \cap P_2 = \{x \mid x \in P_1 \wedge x \in P_2\}$

The Intersection operation produces a new BST containing only the phrases present in both input paragraphs simultaneously.

6.2.1 `void build_intersection_tree(tree_node *p1, tree_node *p2, tree_node **intersection_tree)`

Role: Populates the intersection tree with phrases common to both paragraphs.

Algorithm:

1. Perform an in-order traversal of tree `p1`.
2. For each phrase in `p1`, call `find_phrase()` to search for it in `p2`.

3. If found, insert the phrase into the intersection tree using `insert_copy()`.
4. Otherwise, continue the traversal without inserting.

6.2.2 `tree_node *intersection(tree_node *paragraph1, tree_node *paragraph2)`

Role: Wrapper that manages edge cases and invokes the builder.

Algorithm:

1. If either paragraph is empty (`NULL`), return `NULL` immediately (the intersection of any set with the empty set is empty).
2. Otherwise, create an empty intersection tree and pass it along with both paragraphs to `build_intersection_tree()`.
3. Return the resulting intersection tree.

6.2.3 Intersection — Visual Illustration

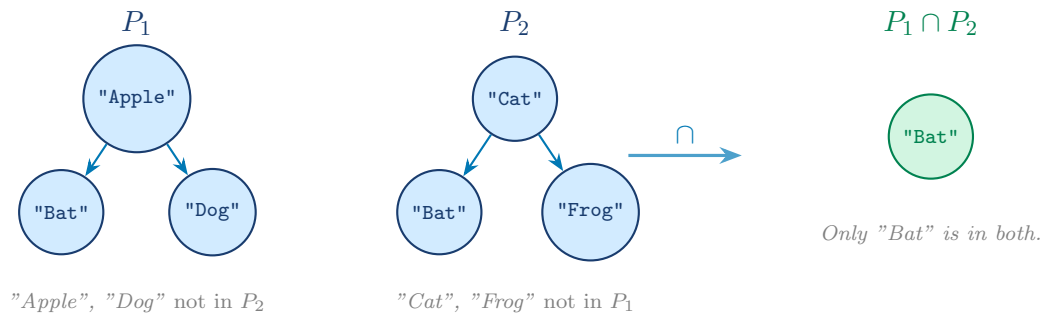


Figure 7: Intersection of two paragraph BSTs. Only phrases found in **both** P_1 and P_2 are retained in the result.

6.3 Difference

Definition: $P_1 \setminus P_2 = \{x \mid x \in P_1 \wedge x \notin P_2\}$

The Difference operation produces a new BST containing only the phrases present in the first paragraph but absent from the second.

6.3.1 `void build_difference_tree(tree_node *p1, tree_node *p2, tree_node **difference_tree)`

Role: Populates the difference tree with phrases exclusive to `p1`.

Algorithm: Follows the same traversal pattern as `build_intersection_tree()`, with the selection criterion inverted: a phrase from `p1` is inserted into the difference tree if and only if `find_phrase()` returns `0` (not found) for that phrase in `p2`.

6.3.2 `tree_node *difference(tree_node *paragraph1, tree_node *paragraph2)`

Role: Wrapper that manages edge cases and invokes the builder.

Algorithm:

1. If `paragraph1` is empty, return `NULL` immediately (the difference of the empty set with anything is empty).
2. Otherwise, create an empty difference tree and pass it along with both paragraphs to `build_difference_tree()`.
3. Return the resulting difference tree.

6.3.3 Difference — Visual Illustration

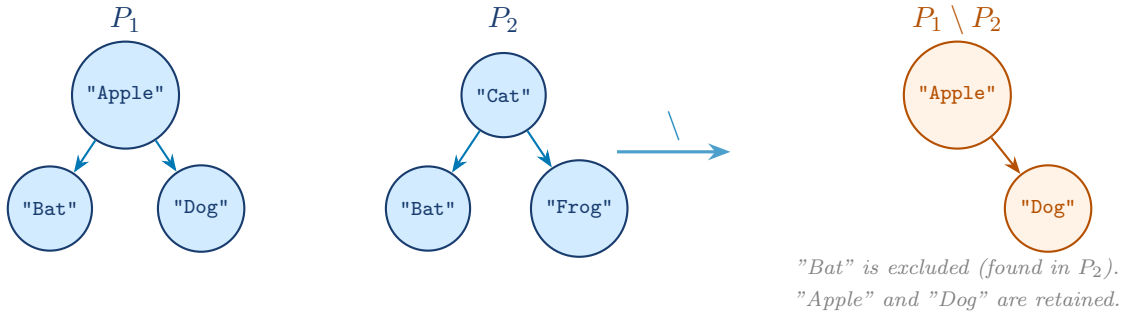


Figure 8: Difference $P_1 \setminus P_2$. Only phrases present in P_1 but absent from P_2 are included in the result tree.

6.3.4 Comparison of Set Operation Behaviours

Phrase	In P_1?	In P_2?	Result (Union / Intersection / Difference)
"Apple"	✓	×	Union: ✓ Inter.: × Diff.: ✓
"Bat"	✓	✓	Union: ✓ Inter.: ✓ Diff.: ×
"Cat"	×	✓	Union: ✓ Inter.: × Diff.: ×
"Dog"	✓	×	Union: ✓ Inter.: × Diff.: ✓
"Frog"	×	✓	Union: ✓ Inter.: × Diff.: ×

7 Program Execution Flow

This section describes the chronological execution of the program from launch to termination, detailing the sequence of function calls and data transformations that occur throughout the lifecycle of the application.

7.1 Initialisation Phase

1. The user is prompted to specify the number of text files to be processed.

2. An array `file[]` of type `text_file` is allocated with the exact required size; no excess capacity is reserved.

7.2 File Processing Phase

For each file i in the array, `process_file()` is called and the resulting `text_file` structure is stored in `file[i]`. The call proceeds as follows:

1. A new `text_file` element is created.
2. `read_file()` is called, which:
   a. Prompts the user to enter the file path.
   b. Sanitises the path (e.g. removes surrounding double quotes).
   c. Opens the file with `fopen()` in read mode (`"r"`).
   d. Loads the entire file content into a heap buffer and returns the starting address.
3. The `index` cursor is initialised to point to the beginning of the buffer.
4. `paragraph_array` is initialised to `NULL` and grows dynamically via `realloc()` as new paragraphs are discovered.
5. For each paragraph, `extract_paragraph()` is called:
   a. A new, empty BST is created for the current paragraph.
   b. `extract_phrase()` is called repeatedly, advancing `index` character by character and building each phrase into a dynamically resizing buffer until a phrase delimiter is reached.
   c. Each extracted phrase is returned to `extract_paragraph()` and inserted into the paragraph's BST via `insert_copy()`.
   d. This loop continues until a newline character (`'\n'`) is encountered, signalling the end of the current paragraph.
6. `paragraph_array` is reallocated to accommodate the new paragraph's BST root pointer. This repeats until the end of the file buffer is reached.
7. `paragraph_count` is set to the total number of paragraphs processed.
8. `process_file()` returns the completed `text_file` structure.

7.3 Operation Phase

Once all files have been processed, the operation menu is displayed. The user selects:

- The desired set operation (Union, Intersection, or Difference).
- The specific paragraphs from specific files to use as operands.

The appropriate pair of BST root pointers (`paragraph_array[i]`) is passed to the selected operation function (`union_op`, `intersection`, or `difference`). The operation executes and returns a result tree.

7.4 Output Phase

The result tree is displayed to the user via `display_inorder()`, which prints all phrases in lexicographic order. The program then terminates.

7.5 Execution Flow Diagram

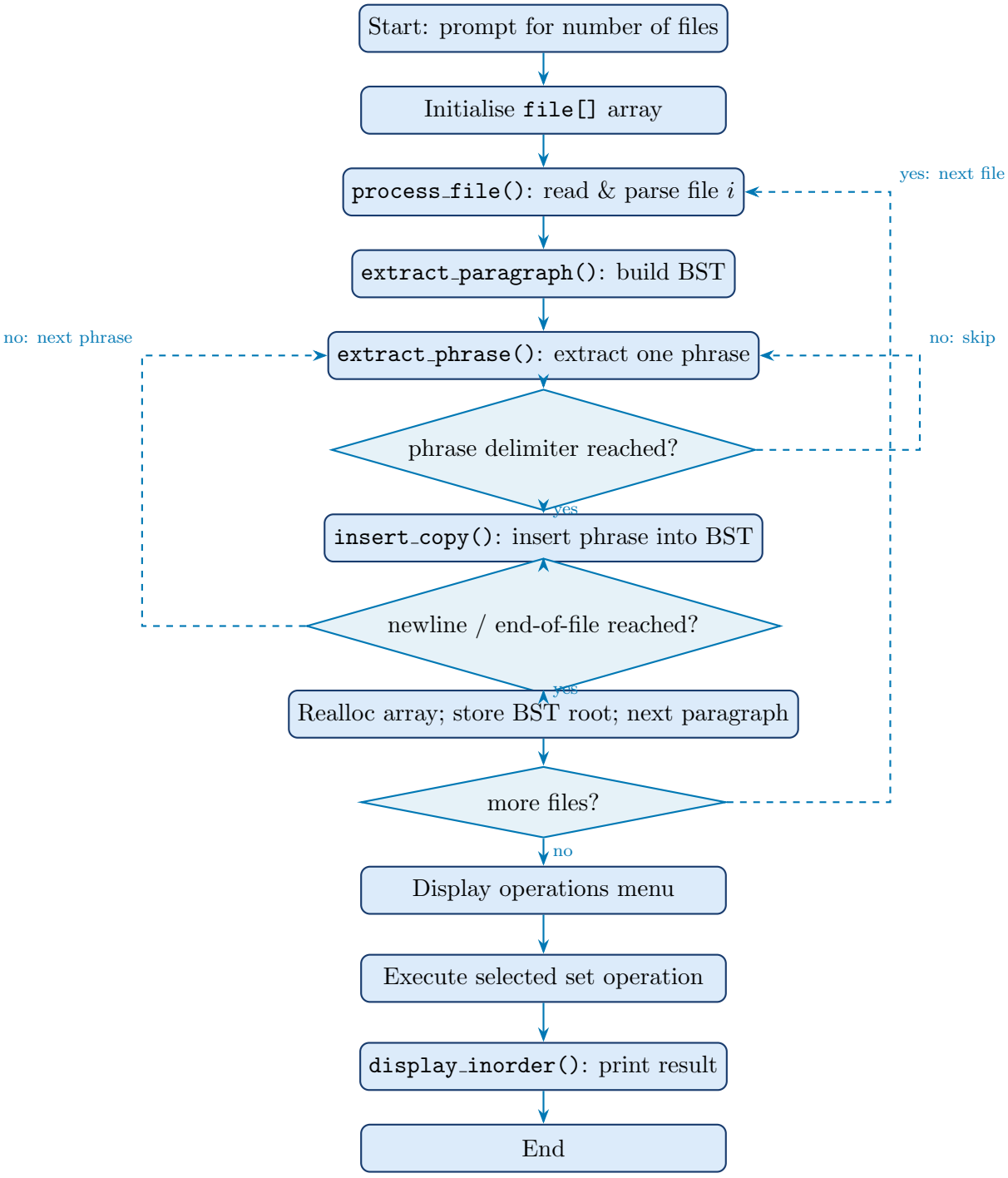


Figure 9: Chronological execution flow of the ADDS application from launch to termination.

8 Complexity Summary

Function	Time Complexity	Notes
`create_phrase()`	$O(1)$	Allocation + pointer assignment
`insert_copy()`	$O(\log n)$	Average case (balanced BST)
`find_phrase()`	$O(\log n)$	Binary search property
`display_inorder()`	$O(n)$	Full traversal
`free_tree()`	$O(n)$	Post-order traversal
`extract_phrase()`	$O(k)$	k = length of extracted phrase
`merge_tree()`	$O(m \log n)$	m = nodes in r_2; n = nodes in r_1
`build_intersection_tree()`	$O(n \log m)$	n = nodes in p_1; m = nodes in p_2
`build_difference_tree()`	$O(n \log m)$	Same as intersection
Paragraph array access	$O(1)$	Direct index access